

Emmanuel Assahou

Mattéo Dabout

Programmation Fonctionnelle

Projet Oral

Partie Business :

Qui n'a jamais connu ce moment exact à l'entraînement où l'on stagne, où les barres refusent de monter, et où le vrai mur n'est plus physique, mais mental et nerveux ? C'est un constat qu'on a personnellement vécu avec mon camarade.

Pour ce projet de programmation fonctionnelle, on a donc décidé de transformer cette frustration en cas d'usage concret : concevoir un véritable assistant de pilotage de la performance pour un athlète cherchant à passer de 67 kg à 70 kg tout en brisant ce fameux palier sur ses charges lourdes au Squat.

Pour y parvenir, notre moteur s'appuie sur des indicateurs scientifiques rigoureux :

- Le **1RM** (*One-Repetition Maximum*), qui mesure la charge absolue maximale qu'un athlète peut soulever sur une seule répétition.
- L'**ACWR** (*Acute-to-Chronic Workload Ratio*), un ratio clé qui compare la fatigue immédiate à la charge de fond pour prévenir instantanément le risque de blessure ou de surentraînement.

Mais en réalité, ce contexte sportif n'est qu'un prétexte. Une belle excuse. Notre véritable objectif aujourd'hui, c'est de vous parler d'ingénierie logicielle et d'architecture. Nous voulons vous démontrer comment la Programmation Fonctionnelle permet de traiter des séries temporelles complexes avec une fiabilité mathématique absolue, par rapport à une approche impérative classique.

Si on avait codé ce projet de manière classique, en orienté objet ou en impératif, on aurait fait ce que tout le monde fait : on aurait initialisé des listes vides, et on aurait fait des boucles for pour ajouter les volumes d'entraînement jour par jour. Le problème de cette approche, c'est **l'état partagé et la mutabilité**. Quand on gère des pipelines de données industriels, que ce soit pour des modèles de pricing en finance ou autres, une variable globale mutée par erreur par une fonction tierce, c'est un cauchemar à débbugger. Les effets de bord détruisent la prévisibilité du code.

Pour y remédier, nous avons imposé des règles strictes à notre base de code, inspirées des standards de la data science moderne :

1. **Zéro mutabilité** : Notre historique de données est figé. Une fois l'entraînement enregistré, l'objet ne peut plus jamais être modifié en place.
2. **Des fonctions 100% pures** : Chaque calcul (tonnage, 1RM, projection) prend une entrée et retourne une nouvelle sortie, sans jamais modifier le contexte global ni interagir avec l'extérieur.
3. **Le bannissement total des boucles for et while** au profit de l'évaluation par flux de données avec map, filter et reduce.

Ces trois règles nous offrent ce qu'on appelle la *transparence référentielle*. Le code devient déterministe, trivial à tester unitairement, et surtout, prêt à être parallélisé.

Partie Code :

Entrons dans le code. Tout part de la donnée. Mon camarade et moi, on a tous les deux déjà ressenti ce palier au squat. Ce moment où tu stagnes, où la charge te fait peur, c'est purement mental et nerveux. Pour analyser ce phénomène sans que notre algorithme ne corrompe les données, nous avons appliqué la première règle de la programmation fonctionnelle : l'immutabilité stricte. En Python, le langage est très permissif. Pour forcer l'immutabilité, nous avons utilisé le décorateur `@dataclass(frozen=True)`. Une fois qu'une séance d'entraînement est instanciée dans notre tuple d'historique, elle est scellée en mémoire. Impossible de faire un `record.weight = 100` par accident.

Mais c'est là que Python avoue ses limites : `frozen=True` est une surcroupe. C'est presque un pansement. Si on regarde notre implémentation en Rust, l'immutabilité n'est pas une option ou un décorateur, c'est l'ADN du langage. En Rust, quand on déclare `let history = load_data();`, la variable est mathématiquement figée par le compilateur. Pour la rendre mutable, il faudrait explicitement écrire `let mut`, ce que nous nous sommes interdits de faire. L'avantage géant de Rust ici, c'est la sécurité garantie à la compilation, essentielle quand on manipule des séries temporelles sensibles.

Une fois la donnée sécurisée, la règle numéro deux était de bannir les boucles `for` et `while`. C'est facile pour faire une somme avec `reduce`, mais ça devient un vrai défi d'ingénierie quand on attaque des modèles mathématiques récurifs, comme l'EWMA (le lissage exponentiel). Pour lisser notre trajectoire de prise de masse vers 70kg et filtrer le "bruit" des pesées quotidiennes, l'EWMA a besoin de l'état du jour précédent. Dans un style impératif, on aurait fait une boucle avec un accumulateur modifiable. Ici, nous avons utilisé `itertools.accumulate`. Cette fonction d'ordre supérieur prend notre flux de poids et y applique une fonction pure (notre `lambda` avec le coefficient `alpha`). Le résultat ? Un flux de données généré dynamiquement, sans jamais réassigner une seule variable.

Et c'est sur ce point précis que Rust écrase littéralement les paradigmes classiques. En Python, chaîner des `filter`, des `map` et des `accumulate` crée souvent de petites listes intermédiaires en mémoire. Ça a un coût. En Rust, on utilise ce qu'on appelle les itérateurs paresseux (Lazy Iterators). Notre lissage EWMA en Rust s'écrit avec `.iter().scan()`. Le côté "geek" et absolument génial ici, c'est ce qu'on nomme la *Zero-Cost Abstraction*. Le compilateur Rust prend toute notre chaîne fonctionnelle et la fusionne en une seule instruction machine ultra-optimisée. On obtient la lisibilité parfaite des mathématiques fonctionnelles, avec une vitesse d'exécution équivalente à du C, sans aucune allocation mémoire superflue. C'est ce qui rend Rust si populaire dans les moteurs de pricing quantitatifs aujourd'hui.

Avant même de penser à l'interface utilisateur, il fallait garantir que nos modèles mathématiques étaient infaillibles. C'est ici que la Règle Numéro 2 du projet — l'utilisation stricte de fonctions pures — prend tout son sens.

Dans une architecture orientée objet ou impérative classique, les tests unitaires sont souvent un cauchemar. Il faut instancier des classes, mocker des bases de données, et surtout écrire

des fonctions complexes de *setup* et *teardown* pour réinitialiser l'état global entre chaque test afin d'éviter qu'ils ne se polluent entre eux.

En programmation fonctionnelle, un test redevient une simple équation mathématique : $f(x) = y$. On prend notre historique figé (notre profil mocké de 67kg), on le passe dans notre fonction `extract_squat_progression`, et on vérifie la sortie. Puisque la fonction est pure et que la donnée d'entrée est immuable, le résultat est 100% déterministe. L'exécution du test A n'altèrera jamais la mémoire du test B. Les "flaky tests" n'existent plus.

C'est une vraie bouffée d'air frais, mais Python nécessite quand même de configurer un framework externe comme `pytest`, de gérer des dossiers séparés et des variables d'environnement. Quand nous avons porté ce moteur de calcul en Rust, nous avons découvert une approche beaucoup plus industrielle. En Rust, le framework de test est intégré directement dans le compilateur. On écrit nos tests unitaires *exactement dans le même fichier* que notre code métier, sous l'annotation `#[cfg(test)]`.

Mais le côté vraiment "geek" et puissant ici, c'est l'intervention du *Borrow Checker* de Rust lors de l'exécution des tests. En Python, si on écrit mal un test unitaire, on peut accidentellement modifier une variable globale qui va faire crasher la suite. En Rust, le compilateur garantit mathématiquement qu'aucune référence mémoire ne peut être mutée silencieusement et partagée entre deux threads de test. Le code métier est validé en isolation totale, à la vitesse du C. C'est exactement cette garantie de fiabilité absolue qui rassure les ingénieurs quantitatifs en salle de marché.

Enfin, la programmation fonctionnelle nous offre un dernier super-pouvoir pour notre frontend Streamlit : la mémoïsation. Streamlit est particulier : à chaque fois que l'utilisateur clique sur un onglet, tout le script Python est réexécuté de haut en bas. Si nos calculs de volume ou d'ACWR faisaient des appels réseau lourds, l'application s'effondrerait. Puisque nous avons respecté la règle des "fonctions pures" (une fonction donne toujours le même résultat pour une même entrée, sans effet de bord), nous avons pu utiliser le décorateur `@st.cache_data`.

Exactement. Streamlit sait que notre fonction est pure. Il calcule le résultat la première fois, le met en cache, et aux exécutions suivantes, il court-circuite le calcul pour renvoyer directement la valeur. En Rust, nous avons appliqué la macro `#[cached]` qui fait exactement la même chose. C'est la beauté de la transparence référentielle : comme le code n'a pas d'effets de bord, le cache devient 100% fiable et mathématiquement sûr.

Partie Conclusion :

Si nous devons pousser ce projet encore plus loin dans le paradigme fonctionnel, la prochaine étape serait de revoir notre gestion des erreurs. Actuellement, en Python, si une donnée manque pour calculer notre EWMA, on risque de lever une exception. Or, les exceptions sont, par définition, des "effets de bord" cachés qui cassent le flux fonctionnel. La solution serait d'implémenter la **Programmation Orientée Rail (Railway-Oriented Programming)** en utilisant des Monades. C'est d'ailleurs ce que Rust fait nativement avec ses types `Option<T>` et `Result<T, E>`. Au lieu de faire crasher le programme, chaque fonction pure retournerait une "boîte" contenant soit la donnée, soit l'erreur, qui se propagerait proprement tout au long du pipeline sans jamais interrompre le flux.

Exactement. Et la deuxième grande ouverture concerne le passage à l'échelle. Puisque nos fonctions sont pures et que nos données (`@dataclass(frozen=True)`) sont immuables, notre code est intrinsèquement *Thread-Safe*. Si demain nous devons analyser non plus l'historique d'un seul athlète, mais les données biomécaniques de 10 000 utilisateurs en temps réel, nous pourrions paralléliser ces calculs sur plusieurs cœurs instantanément. En Rust, l'absence d'effets de bord garantit qu'il n'y aura aucune *data race*. En Python, cela nous permettrait de brancher notre logique directement sur des outils distribués comme Apache Spark, dont l'architecture repose justement sur ces mêmes concepts fonctionnels (map, reduce)